

《计算机视觉》实验报告

姓名：周梓俊 学号：23120899

实验 3

一、任务 1

a) 核心代码：

```
1  # 转换为HSV空间
2  hsv = cv2.cvtColor(img, cv2.COLOR_BGR2HSV)
3
4  # 分离RGB通道和HSV通道
5  b, g, r = cv2.split(img)
6  h_ch, s_ch, v_ch = cv2.split(hsv)
7
8  # 对RGB三个通道画三维图
9  rows = np.arange(0, img.shape[0])
10 cols = np.arange(0, img.shape[1])
11 X, Y = np.meshgrid(cols, rows)
12
13 step = max(1, max(img.shape[0], img.shape[1]) // 100)
14 X_s = X[::step, ::step]
15 Y_s = Y[::step, ::step]
16
17 ax1 = fig.add_subplot(131, projection='3d')
18 ax1.plot_surface(X_s, Y_s, r[::step, ::step], cmap='Reds')
19
20 ax2 = fig.add_subplot(132, projection='3d')
21 ax2.plot_surface(X_s, Y_s, g[::step, ::step], cmap='Greens')
22
23 ax3 = fig.add_subplot(133, projection='3d')
24 ax3.plot_surface(X_s, Y_s, b[::step, ::step], cmap='Blues')
```

b) 实验结果截图



R 通道



G 通道



B 通道



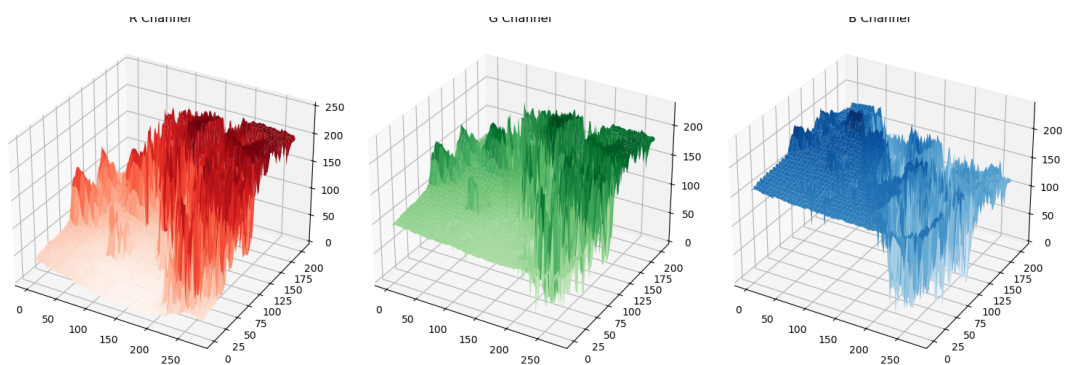
H 通道



S 通道



V 通道



RGB 三通道三维图

c) 实验小结

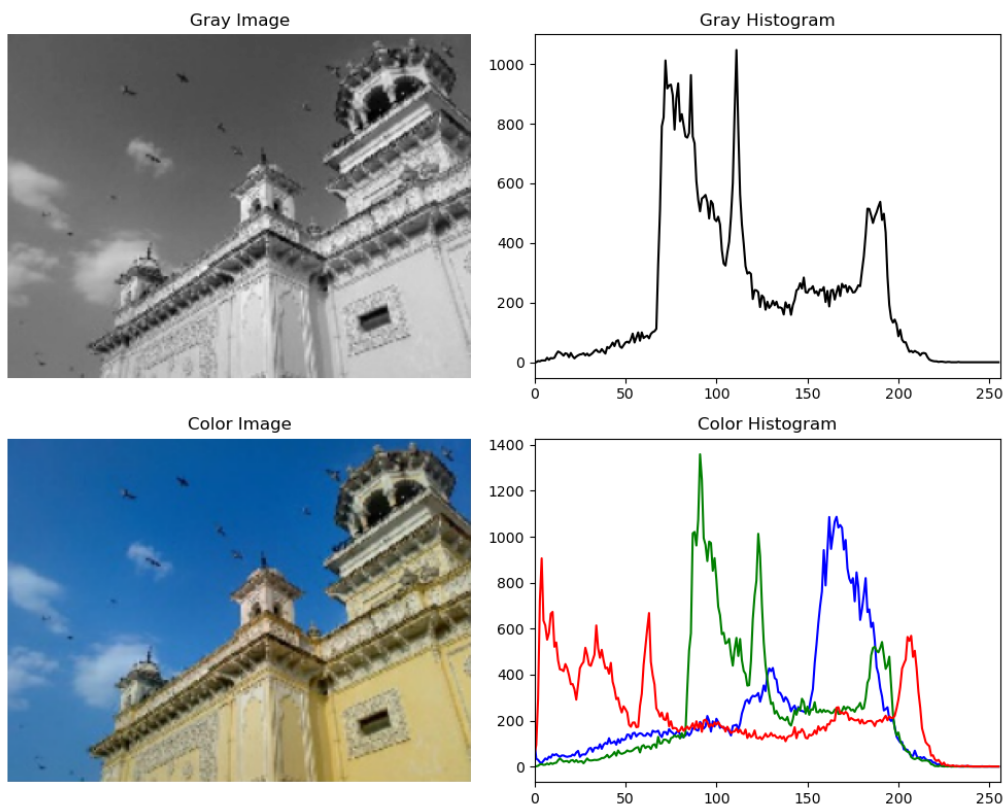
在任务 1 中，我首先将图像从 BGR 颜色空间转换为 HSV 颜色空间，然后使用 `cv2.split` 分别分离了 RGB 三个通道和 HSV 三个通道并显示。接着利用 `matplotlib` 的 `plot_surface` 函数对 R、G、B 三个通道分别绘制了三维表面图，其中 `x` 和 `y` 轴表示像素坐标，`z` 轴表示该通道的像素值。

二、任务 2

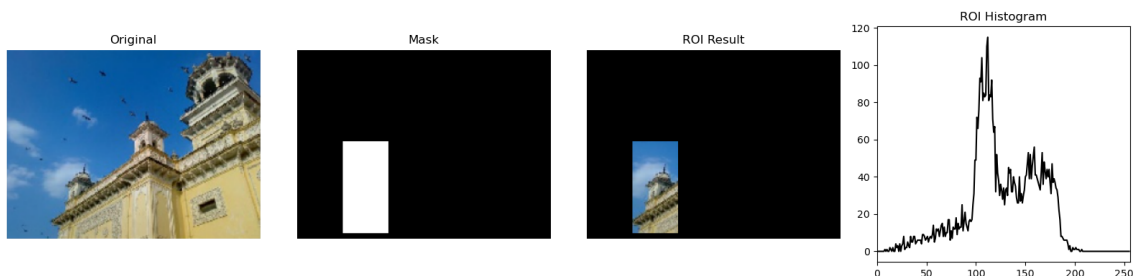
a) 核心代码:

```
1 home_color = cv2.imread("home_color.png")
2 home_gray = cv2.cvtColor(home_color, cv2.COLOR_BGR2GRAY)
3
4 # 灰度直方图
5 gray_hist = cv2.calcHist([home_gray], [0], None, [256], [0, 256])
6
7 # 彩色直方图 (B/G/R三通道分别计算)
8 colors = ('b', 'g', 'r')
9 for i, col in enumerate(colors):
10     hist = cv2.calcHist([home_color], [i], None, [256], [0, 256])
11     axes[1, 1].plot(hist, color=col)
12
13 # ROI直方图, ROI区域 x:50-100, y:100-200
14 mask = np.zeros(home_gray.shape[:2], dtype=np.uint8)
15 mask[100:200, 50:100] = 255
16 roi_img = cv2.bitwise_and(home_color, home_color, mask=mask)
17 roi_hist = cv2.calcHist([home_gray], [0], mask, [256], [0, 256])
```

b) 实验结果截图



灰度直方图与彩色直方图



ROI 区域：原图、Mask、ROI 提取结果、ROI 直方图

c) 实验小结

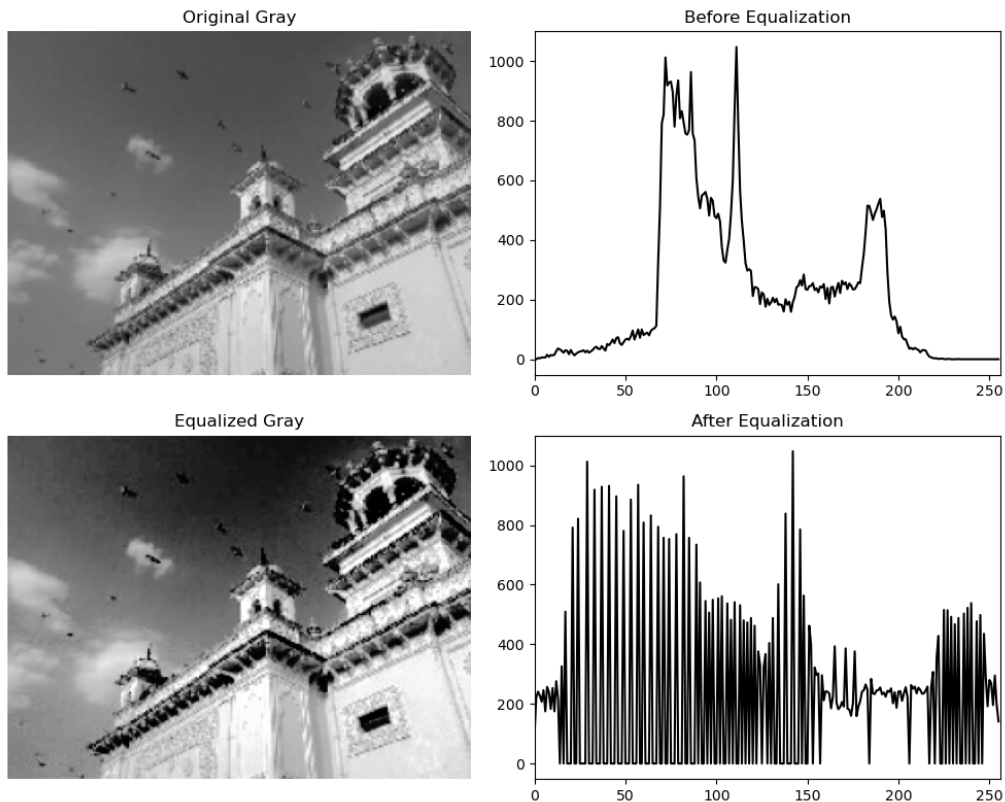
在任务 2 中，我使用 `cv2.calcHist` 函数分别计算了灰度图的直方图和彩色图的 BGR 三通道直方图，并将原图与对应直方图拼接在同一个窗口中显示。对于 ROI 区域，通过创建一个指定区域为白色的 `mask`，利用 `bitwise_and` 提取了感兴趣区域的图像，并计算了该区域的灰度直方图。最终将原图、`mask` 图、ROI 提取结果和 ROI 直方图放在同一个窗口内展示。

三、任务 3

a) 核心代码:

```
1 gray3 = cv2.cvtColor(img3, cv2.COLOR_BGR2GRAY)
2
3 # 直方图均衡化
4 equalized = cv2.equalizeHist(gray3)
5
6 # 计算均衡化前后的直方图
7 hist_before = cv2.calcHist([gray3], [0], None, [256], [0, 256])
8 hist_after = cv2.calcHist([equalized], [0], None, [256], [0, 256])
```

b) 实验结果截图



直方图均衡化前后对比

c) 实验小结

在任务 3 中, 我使用 `cv2.equalizeHist` 函数对灰度图像进行了直方图均衡化处理。均衡化前的直方图像素值分布集中在某些区间, 均衡化后像素值分布更加均匀, 图像的对比度得到了明显提升。